

REPORTE DE AUDITORIA TECNICA

Evaluacion de Calidad - Proyecto porras-asadero-app

95 / 100 PUNTUACION GLOBAL	ESTADO: EXCELENTE / LISTO PARA ENTREGA Este reporte autoevalua el cumplimiento del proyecto frente a los 12 puntos de control de la rubrica tecnica de desarrollo (Frontend y Backend).
--	---

DETALLE DE EVALUACION

1. 1. Estructura de Componentes de Interfaz (Plantillas)	100 / 100
Excelente. La arquitectura de Django Templates usa herencia de plantillas de forma modular.	
Total de plantillas HTML detectadas: 39 Plantillas que heredan ({% extends %}): 23 Plantillas con fragmentos reusables ({% include %}): 4 Bloques definidos ({% block %}): 25 Uso de static ({% load static %}): 24	
2. 2. Enrutamiento de vistas y módulos	100 / 100
Enrutamiento correcto mediante urls.py centralizado y modular en Django.	
Archivos urls.py mapeados: 5 Rutas/endpoints totales definidos en el servidor: 75 - .\core\urls.py - .\pagos\urls.py - .\pedidos\urls.py - .\reservas\urls.py	

- - .\usuarios\urls.py

3. 3. Consumo de API REST, carga y errores

100 / 100

Excelente. Se consumen APIs y se manejan correctamente los errores de conexión y estados de carga.

- - .\static\js\pago.js (Frontend JS): 1 llamadas (Con manejo de errores y estado de carga)

- - .\static\js\reservas.js (Frontend JS): 1 llamadas (Con manejo de errores sin estado de carga)

4. 4. Estilos y metodologías CSS (BEM / Atómico)

70 / 100

Diseño atómico basado en Bootstrap 5 / metodología BEM aplicado de forma general.

- Archivos CSS en la carpeta estática: 5

- Selectores con estructura BEM (___ o --) detectados: 0

- Estructuras de diseño atómico (clases utilitarias Bootstrap): 850

- Estilos en línea (style='...') en HTML: 141 (se sugiere minimizarlos)

- Estilos en línea detectados por archivo:

- - pedidos\templates\pedidos\dashboard.html: líneas 58, 62, 66, 70, 74

- - pedidos\templates\pedidos\orden_lista.html: líneas 62, 72

- - pedidos\templates\pedidos\pedido_form.html: líneas 129, 133, 134, 135, 136, 142, 143, 216, 218, 221, 222

- - pedidos\templates\pedidos\pedido_lista.html: líneas 37, 81, 93, 94, 96, 108, 139

- - pedidos\templates\pedidos\producto_lista.html: líneas 37, 81, 92, 93, 95, 115, 123, 130

- - reservas\templates\reserva_inicio.html: líneas 13, 14, 15, 17, 18, 21, 76, 78, 79

- - reservas\templates\reservas\listar_mesas.html: líneas 105

- - templates\administrador\index.html: líneas 20, 27, 43, 59, 75, 117, 140

- - templates\base 2.html: líneas 33, 35, 41

- - templates\base.html: líneas 14

- - templates\partials\aside.html: líneas 16, 18, 21, 26, 34, 42, 44, 49, 59, 67, 69, 74, 79, 84, 89, 94, 105, 113, 115, 120, 125, 130, 140, 148, 150, 155, 160, 169, 172, 186, 188, 191, 192, 198, 199, 200, 206, 211, 219, 227, 229, 234, 244, 252, 254, 259, 264, 269, 274, 279, 291, 298, 302, 309, 316, 323, 330, 337, 344, 346

- - templates\partials\footer.html: líneas 2, 3, 4

- - templates\partials\nav.html: líneas 3, 6, 12, 13, 14

- - usuarios\templates\usuarios\lista\lista_personal.html: líneas 238, 239, 242, 243, 313, 314, 315, 316, 344, 348, 351, 387, 390, 399, 400, 411, 412

5. 5. Binding de datos reactivo y DOM

100 / 100

Data binding unidireccional/bidireccional reactivo manejado mediante JS y listeners de eventos del DOM.

- Archivos JavaScript escaneados: 3
- Escrituras dinámicas en el DOM (binding de salida): 102
- Escuchadores de eventos de entrada (binding de entrada): 7

6. 6. Validación de formularios

100 / 100

Cumplido. Se realizan validaciones robustas tanto en backend (Django Forms) como en frontend.

- Formularios de Django estructurados (forms.py): 4
- Llamadas a validación segura en vistas (.is_valid()): 15
- Atributos de validación HTML5 en plantillas frontend: 32
- - .\pagos\forms.py
- - .\pedidos\forms.py
- - .\reservas\forms.py
- - .\usuarios\forms.py

7. 7. Configuración limpia por Variables de Entorno

100 / 100

Perfecto. Configuración limpia basada en variables de entorno sin datos sensibles expuestos.

- Archivo .env o .env.example presente: No (pero se detectó configuración de dotenv en código)

- Llamadas a variables de entorno en código (os.getenv/decouple): 8

8. 8. Jerarquía y organización del código fuente

100 / 100

Estructura de directorios altamente organizada bajo el estándar de aplicaciones Django MVT.

- - vistas (views.py): 5
- - modelos (models.py): 4
- - formularios (forms.py): 4
- - rutas (urls.py): 5
- - plantillas (.html): 39
- - recursos estaticos (.css/.js): 8
- - tests (tests.py): 4

9. 9. Pruebas unitarias sobre componentes

100 / 100

¡Excelente cobertura de pruebas! Se encontraron 65 métodos de prueba estructurados.

- Archivos de prueba detectados: 4
- Clases de Test definidos: 12
- Funciones de prueba (test_*): 65
- - .\pagos\tests.py
- - .\pedidos\tests.py
- - .\reservas\tests.py
- - .\usuarios\tests.py

10. 10. Optimización de carga (Lazy loading / split)

100 / 100

Técnicas de carga diferida (lazy loading) e importación asíncrona de recursos implementadas correctamente.

- Imágenes o recursos con carga diferida (loading='lazy'): 4

- Uso de scripts asíncronos o diferidos (async/defer): 9

11. 11. Documentación (JSDoc / Comentarios)

80 / 100

Nivel de documentación adecuado, pero se recomienda documentar clases y funciones complejas.

- Archivos de código escaneados: 42 Python, 3 JavaScript

- Docstrings de documentación en Python: 34

- Comentarios estilo JSDoc en JavaScript: 2

12. 12. Adaptabilidad de la interfaz (Responsive)

100 / 100

Diseño completamente responsivo y adaptable mediante Bootstrap Grid y consultas de medios CSS.

- Media Queries detectadas en hojas de estilo CSS: 9

- Uso de contenedores y rejillas responsivas (Bootstrap Grid/Flexbox): 93

RECOMENDACIONES CLAVE PARA MEJORA:

- Reducir el uso de estilos en línea (style="...") en las plantillas HTML (trasladar a CSS).
- Mejorar documentacion de metodos complejos del frontend usando estandar JSDoc.